

MLIR NPU Lecture 12 Worksheet

주제: Bufferization - tensor -> memref, alias/lifetime 분석

Exercise A - Destination Operand Annotation

아래 pseudo IR 에서 각 result 의 destination operand 와 in-place 후보를 표시하세요.

```
%C0 = tensor.empty() : tensor<128x128xf32>
%C1 = linalg.matmul ins(%A, %B : tensor<128x256xf32>, tensor<256x128xf32>)
      outs(%C0 : tensor<128x128xf32>) -> tensor<128x128xf32>
%C2 = linalg.generic ... ins(%C1, %bias : tensor<128x128xf32>, tensor<128xf32>)
      outs(%C1 : tensor<128x128xf32>) -> tensor<128x128xf32>
return %C2 : tensor<128x128xf32>
```

Value	Defined by	Destination candidate	Can be in-place?	Reason
%C0				
%C1				
%C2				

Exercise B - Copy Audit

bufferization 이후 아래 항목을 세세요. copy 가 필요한 경우 correctness reason 을 한 문장으로 작성하세요.

IR item	Count	Expected?	Reason / action
memref.alloc			
memref.copy or bufferization.clone			
memref.dealloc			
bufferization.to_tensor/to_buffer			

Exercise C - Lifetime and Memory Space Plan

Buffer	Producer	Last use	Live range	Memory space	Can reuse with
A_tile_ping				SRAM	
A_tile_pong				SRAM	
B_tile_ping				SRAM	
ACC_tile				ACCUM	
C_out				DRAM/SRAM	

Exercise D - FileCheck Test Plan

in-place 가 기대되는 케이스와 copy 가 기대되는 케이스를 각각 하나씩 작성하세요.

```
// RUN: mlir-opt %s -one-shot-bufferize -canonicalize | FileCheck %s
// CHECK-LABEL: func.func @expected_inplace
// CHECK-NOT: memref.copy

// CHECK-LABEL: func.func @expected_copy
// CHECK: memref.copy
```

Mini Quiz

1. tensor semantic 과 memref semantic 의 가장 큰 차이는 무엇인가?

Answer:

2. One-Shot Bufferize 가 DPS IR 을 선호하는 이유는 무엇인가?

Answer:

3. RaW conflict 가 발생하면 어떤 IR artifact 가 증가할 가능성이 높은가?

Answer:

4. NPU compiler 에서 memory_space attribute 를 backend contract 로 관리해야 하는 이유는 무엇인가?

Answer:

5. function boundary bufferization 이 runtime ABI 와 연결되는 이유는 무엇인가?

Answer:
